

Helix OTA — As-Built Operational & Rollout REST Endpoints (/api/v1)

Field	Value
Revision	2
Last modified	2026-06-08T00:00:00Z
Created	2026-06-08
Status	active (as-built — generated from the real route table) An as-built reference for the operational + staged-rollout REST endpoints actually wired in <code>server/internal/api/server.go</code> (delta artifacts register+find, the audit read, telemetry reads, device-group CRUD + membership, the staged-rollout create/get/evaluate routes, and the server-driven recall + rollback-history routes). It documents only routes that exist in the code and cites the handler file: func each path maps to. It is the implemented counterpart to the prose specs <code>operational_endpoints.md</code> (audit/telemetry/groups), <code>delta_updates_design.md</code> (deltas), <code>1.0.1-staged-rollout/rollout_engine.md</code> (rollout), and <code>../1.0.1-staged-rollout/rollback_ux.md</code> §7 (recall); where the implementation and the spec disagree, the divergence is called out in §11. The operator recall endpoint is now WIRED (<code>handlers_recall.go</code>), no longer planned. <code>server/internal/api/server.go</code> route table — <code>Router()</code>, lines 111–183 (specifically the protected auth group, lines 132–179, plus the unversioned probes on lines 118–119). This document was hand-authored from a direct read of that route table and the handler sources; it is not auto-generated, but every row traces to a cited line/function.
Status summary	
Generated from	

Field	Value
Anti-bluff	Per Constitution §11.4.6 (no-guessing): only routes present in <code>server.go</code> are documented. Status/Action/Reason enum values are taken from the <code>ota-rollout-engine</code> brick source (<code>types.go</code> , <code>verdict.go</code>), not invented. Spec/impl divergences are listed in §11, never silently reconciled.
Owner	Helix OTA control-plane team
Related	endpoints.md (parent REST spec — conventions, error model, RBAC); operational_endpoints.md (audit/telemetry/groups prose spec); openapi.yaml ; ../1.0.1-staged-rollout/rollout_engine.md ; ../1.0.1-staged-rollout/rollback_ux.md

table_of_contents

- [purpose_and_scope](#)
- [how_to_read_this_document](#)
- [as_built_endpoint_table](#)
- [audit_read_endpoint](#)
- [telemetry_read_endpoints](#)
- [device_group_crud_and_membership](#)
- [staged_rollout_endpoints](#)
- [recall_and_rollback_history_endpoints](#)
- [delta_artifact_endpoints](#)
- [audit_write_middleware_cross_cutting](#)
- [spec_vs_implementation_divergences](#)

1. purpose_and_scope

This is the **as-built** reference for the operational and staged-rollout endpoints wired this session into the Helix OTA control plane. It documents, for each route that **actually exists in `server/internal/api/server.go`**:

- HTTP method + path (relative to the `/api/v1` base);
- required roles (from the literal `requireRole(...)` call on that route in `server.go`);
- request body type and response type + key fields (from the handler + wire-type sources);
- status codes the handler can emit;
- the handler `file:func` the route maps to.

Conventions (base path `/api/v1`, the `{ "error": { code, message, request_id, details[] } }` envelope, RBAC roles `admin/operator/viewer/device`, transport/compression, pagination header `?limit/?cursor`) are **inherited from [endpoints.md](#)** and not re-specified here.

In scope (all confirmed present in the route table):

- **Delta artifacts** — `POST /deltas` and `GET /deltas` (`server.go:140–141`).
- **Audit read** — `GET /audit` (`server.go:179`).
- **Telemetry reads** — `GET /devices/:deviceId/telemetry` (`server.go:164`) and `GET /telemetry/overview` (`server.go:165`).
- **Device-group CRUD + membership** — `POST/GET/GET/PATCH/DELETE /groups[...]` and the members sub-routes (`server.go:169–176`).
- **Staged rollout** — `POST/GET /deployments/:deploymentId/rollout` and `POST /deployments/:deploymentId/rollout/evaluate` (`server.go:152–154`).
- **Recall + rollback history** — `POST /deployments/:deploymentId/recall` and `GET /deployments/:deploymentId/rollbacks` (`server.go:157–158`).

Out of scope: the auth, device-register, artifact, release, deployment-CRUD, and client (device) endpoints — those are the 1.0.0-MVP base surface documented in [endpoints.md](#).

2. how_to_read_this_document

Each endpoint section states the **verified** facts pulled from source:

- **Roles** are the exact arguments to `requireRole(...)` in `server.go` — the codebase expresses the role hierarchy by listing all accepted roles (e.g. `requireRole(RoleViewer, RoleOperator, RoleAdmin)`), so `admin  $\supseteq$  operator  $\supseteq$  viewer` is realized by enumeration, not inheritance.
- **Request / Response types** name the Go wire structs (and their JSON field tags) in the handler files; the JSON examples below are rendered from those struct tags.
- **Maps to** gives `relative/handler.go:HandlerFunc`.

Where the implemented shape differs from the prose spec it implements, the difference is noted inline and collected in [§11](#).

3. as_built_endpoint_table

Method	Path (<code>/api/v1 + ...</code>)	Required roles	Request type	Response type
POST	<code>/deltas</code>	operator, admin	<code>DeltaRegister</code>	<code>DeltaView</code>
GET	<code>/deltas</code>	viewer, operator, admin	— (query params)	<code>DeltaView</code>
GET	<code>/audit</code>	admin	— (query params)	<code>AuditLogList</code>

Method	Path (/api/v1 + ...)	Required roles	Request type	Response type
GET	/devices/:deviceId/telemetry	viewer, operator, admin, device (own id)	—	TelemetryHistory
GET	/telemetry/overview	viewer, operator, admin	—	TelemetryOverview
POST	/groups	operator, admin	GroupCreate	GroupView
GET	/groups	viewer, operator, admin	—	GroupList
GET	/groups/:groupId	viewer, operator, admin	—	GroupView
PATCH	/groups/:groupId	operator, admin	GroupUpdate	GroupView
DELETE	/groups/:groupId	admin	—	— (204)
GET	/groups/:groupId/members	viewer, operator, admin	—	GroupMembers
POST	/groups/:groupId/members	operator, admin	MemberAdd (batch)	MemberAddResult (200)
DELETE	/groups/:groupId/ members/:deviceId	operator, admin	—	— (204)
POST	/deployments/:deploymentId/ rollout	operator, admin	RolloutCreate	RolloutState
GET	/deployments/:deploymentId/ rollout	viewer, operator, admin	—	RolloutState
POST	/deployments/:deploymentId/ rollout/evaluate	operator, admin	RolloutVerdict	RolloutDecision
POST	/deployments/:deploymentId/ recall	operator, admin	RecallRequest	RollbackView
GET	/deployments/:deploymentId/ rollbacks	viewer, operator, admin	—	RollbackList

The path param uses the gin :name form in the route table; the public-facing spec uses {name}. They are the same parameter.

4. audit_read_endpoint

4.1 GET /api/v1/audit

Reads the audit trail. Admin-only.

- **Roles:** admin (requireRole(RoleAdmin), server.go:179).
- **Maps to:** handlers_audit.go:handleListAudit.
- **Request:** no body. Query parameters (all optional, combined as AND):
 - ?action= — exact action verb filter.
 - ?resource_type= — exact resource-type filter.
 - ?cursor= — opaque pagination cursor.
 - ?limit= — integer in [1, 200] (default 50). Out of range → 400 VALIDATION_FAILED with details: [{field:"limit", issue:"out of range"}].
 - ?since= — RFC3339 lower time bound (store.AuditFilter.Since). A non-RFC3339 value → 400 VALIDATION_FAILED with details: [{field:"since", issue:"not RFC3339"}].
 - ?until= — RFC3339 upper time bound (store.AuditFilter.Until). A non-RFC3339 value → 400 VALIDATION_FAILED with details: [{field:"until", issue:"not RFC3339"}].
- **Response 200** (AuditLogList, audit_wire.go):

```
{
  "items": [
    {
      "id": "9c2f...",
      "actor": "admin@example.com",
      "action": "DEPLOYMENT_CREATE",
      "resource_type": "deployment",
      "resource_id": "d12b...",
      "details": { "version": "1.1.0" },
      "ip_address": "203.0.113.7",
      "user_agent": "helix-dashboard/1.0",
      "created_at": "2026-06-08T10:15:00Z"
    }
  ],
  "next_cursor": null
}
```

- **Key fields** (AuditLogEntry): id, actor (a **flat string** — the actor subject, falling back to the user id; audit_wire.go:toAuditLogEntry), action, resource_type, resource_id (omitempty), details (string→string map, oitempty), ip_address (omitempty), user_agent (omitempty), created_at. next_cursor is *string (JSON null when exhausted).
- **Status codes:** 200 OK; 400 VALIDATION_FAILED (bad limit, or non-RFC3339 since/until); 500 INTERNAL (store error → "could not list audit log"). Auth failures (401/403) are produced by the shared authMiddleware + requireRole ahead of the handler.
- **Immutability:** read-only over the API — there is no POST/PATCH/DELETE / audit. Rows are written only by the audit middleware (§10).

5. telemetry_read_endpoints

5.1 GET /api/v1/devices/{deviceId}/telemetry

Per-device telemetry event history.

- **Roles:** viewer, operator, admin, device (server.go:164). A device token may read **only its own** id: the handler compares claims. Subject to the path deviceId and returns 403 FORBIDDEN ("a device may read only its own telemetry") for a non-privileged caller reading another device.
- **Maps to:** handlers_telemetry.go:handleDeviceTelemetry.
- **Request:** no body, no query parameters consumed by the handler.
- **Response 200** (TelemetryHistory):

```
{
  "device_id": "8f3a...",
  "events": [
    {
      "event": "download_started",
      "version": "1.1.0",
      "deployment_id": "d12b...",
      "error_code": "",
      "detail": "",
      "timestamp": "2026-06-08T10:15:00Z",
      "received_at": "2026-06-08T10:15:01Z"
    }
  ]
}
```

- **Key fields** (TelemetryEventView): event (otaprotocol.TelemetryEvent), version, deployment_id, error_code, detail (all omit empty), timestamp, received_at. Events are returned in insertion order (the store contract TelemetryForDevice, store.go:179–181).
- **Status codes:** 200 OK; 403 FORBIDDEN (cross-device device token); 500 INTERNAL ("could not read telemetry").

5.2 GET /api/v1/telemetry/overview

Fleet-wide telemetry counts grouped by event type.

- **Roles:** viewer, operator, admin (server.go:165). No device access.
- **Maps to:** handlers_telemetry.go:handleTelemetryOverview.
- **Request:** no body, no query parameters.
- **Response 200** (TelemetryOverview):

```
{
  "event_counts": {
    "download_started": 1188,
    "installing": 1152,
    "success": 1100,
    "failure": 36
  }
}
```

```

},
"total": 3476,
"failure_rate": 0.0317,
"by_state": {
  "idle": 980,
  "updating": 42,
  "failed": 12
}
}

```

- **Key fields** (TelemetryOverview): event_counts (string→int64 map, keyed by event type via the store TelemetryEventCounts), total (sum of all counts, computed in the handler), failure_rate (float64) and by_state (string→int64 map). failure_rate is computed in the handler as failure / (success + failure) — the fraction of failed terminal outcomes among the two terminal events (EventSuccess, EventFailure); it is 0 when there are no terminal events yet. by_state is the fleet device count keyed by last-known update state, from the store DeviceStateCounts. (Both fields landed this session — see §11.)
- **Status codes:** 200 OK; 500 INTERNAL ("could not aggregate telemetry" on the event-count read, or "could not aggregate device states" on the device-state read).

6. device_group_crud_and_membership

Wire types: handlers_group.go. Store seam: store.go (CreateGroup, GetGroup, ListGroups, UpdateGroup, DeleteGroup, AddGroupMember, ListGroupMembers, RemoveGroupMember).

6.1 POST /api/v1/groups

- **Roles:** operator, admin (server.go:169). **Maps to:** handleCreateGroup.
- **Request body** (GroupCreate): { "name": "...", "description": "...". } name is **required** — blank → 400 VALIDATION_FAILED (details: [{field:"name",issue:"required"}]).
- **Response 201** (GroupView): { "group_id", "name", "description"(omitempty), "member_count", "created_at" } (member_count is the live membership count; 0 for a just-created group).
- **Status codes:** 201 Created; 400 VALIDATION_FAILED (malformed body / blank name); 409 CONFLICT (store.ErrConflict → "a group with that name already exists"); 500 INTERNAL.

6.2 GET /api/v1/groups, GET /api/v1/groups/{groupId}

- **Roles:** viewer, operator, admin (server.go:170–171).
- **List** (handleListGroups): response GroupList { "items": [GroupView] } — **no pagination** (ListGroups takes no filter/cursor; 500 INTERNAL on store error).
- **Read one** (handleGetGroup): response GroupView; 404 NOT_FOUND ("group not found") if absent.

6.3 PATCH /api/v1/groups/{groupId}, DELETE /api/v1/groups/{groupId}

- **PATCH** (handleUpdateGroup, server.go:172): roles operator, admin. Body GroupUpdate { "name", "description" }. The handler loads the existing group (404 if absent), applies name only when non-empty, and **always** sets description from the body (a missing description clears it — see §11). Response 200 GroupView. 409 CONFLICT on name collision; 404 NOT_FOUND; 500 INTERNAL.
- **DELETE** (handleDeleteGroup, server.go:173): role admin only. Response 204 No Content. Any store error maps to 404 NOT_FOUND ("group not found").

6.4 membership endpoints

- **GET /groups/{groupId}/members** (handleListGroupMembers, server.go:174): roles viewer, operator, admin. Response GroupMembers { "group_id", "device_ids": [...] } (empty array, never null). 404 NOT_FOUND if the group is missing.
- **POST /groups/{groupId}/members** (handleAddGroupMembers, server.go:175): roles operator, admin. Body MemberAdd { "device_ids": [...] } — a **batch** (non-empty, blank/empty → 400 VALIDATION_FAILED). Response 200 (MemberAddResult) { "added": [...], "already_member": [...], "not_found": [...] } — a device must be REGISTERED to be added (unregistered ids → not_found), ids already in the group → already_member, the rest → added (duplicates within the batch de-duped). 404 NOT_FOUND if the GROUP is missing; 500 INTERNAL.
- **DELETE /groups/{groupId}/members/{deviceId}** (handleRemoveGroupMember, server.go:176): roles operator, admin. Response 204 No Content. 404 NOT_FOUND (missing group); 500 INTERNAL.

7. staged_rollout_endpoints

Wire types: handlers_rollout.go. Engine: the ota-rollout-engine brick (github.com/ HelixDevelopment/ota-rollout-engine), driven via server/internal/rollout.Service(s.rollout, wired in server.go:96). Enum values below are taken from the brick source (types.go, verdict.go).

7.1 POST /api/v1/deployments/{deploymentId}/rollout

Create + start a staged rollout for an existing deployment.

- **Roles:** operator, admin (server.go:152). **Maps to:** handleCreateRollout.
- **Request body** (RolloutCreate): { "phases": [RolloutPhaseSpec, ...] }, **at least one** phase (empty → 400 VALIDATION_FAILED, details: [{field:"phases",issue:"must not be empty"}]). Each RolloutPhaseSpec:

```
{  
  "percentage": 10,  
  "success_threshold": 0.95,  
}
```



```

    "error_threshold": 0.02,
    "duration_seconds": 3600,
    "auto_progress": true
}

```

duration_seconds is converted to a time.Duration for the engine

Phase.Duration. - **Pre-check:** the deployment must exist

(s.repo.GetDeployment) → else 404 NOT_FOUND ("deployment not found"). -

Response 201 (RolloutState):

```

{
  "deployment_id": "d12b...",
  "status": "active",
  "current_phase": 0,
  "phases": [ { "percentage": 10, "success_threshold": 0.95,
                "error_threshold": 0.02, "duration_seconds": 3600,
                "auto_progress": true } ],
  "updated_at": "2026-06-08T10:30:00Z"
}

```

- **status** is the engine Status string. Closed set (brick types.go): pending, active, halted, completed, held.
- **Status codes:** 201 Created; 404 NOT_FOUND (deployment missing); 400 VALIDATION_FAILED (malformed body, empty phases, or the brick's plan validation failing — the handler maps a CreateAndStart error to 400 with the brick error string in details, e.g. percentages must be strictly increasing and end at 100, thresholds in [0,1]).

7.2 GET /api/v1/deployments/{deploymentId}/rollout

- **Roles:** viewer, operator, admin (server.go:153). **Maps to:** handleGetRollout.
- **Response 200** (RolloutState, same shape as §7.1).
- **Status codes:** 200 OK; 404 NOT_FOUND ("no rollout for this deployment" — when Service.Get returns the engine's not-found).

7.3 POST /api/v1/deployments/{deploymentId}/rollout/evaluate

Apply a telemetry-derived health verdict to the current phase and return the engine decision.

- **Roles:** operator, admin (server.go:154). **Maps to:** handleEvaluateRollout.
- **Request body** (RolloutVerdict):

```

{ "success_rate": 0.97, "error_rate": 0.01,
  "post_boot_health_failed": false }

```

- **Response 200** (RolloutDecision):

```

{
  "action": "advance",
  "reason": "success_threshold_met",

```

```

"state": { "deployment_id": "d12b...", "status": "active",
           "current_phase": 1, "phases": [], "updated_at": "..."}
}

```

- **action** — engine Action (brick verdict.go): halt, advance, hold, complete.
- **reason** — engine Reason (brick verdict.go): error_threshold_breached, post_boot_health_failed, success_threshold_met, evaluation_window_open, window_expired_below_threshold, auto_progress_disabled, no_active_phase.
- **state** — the post-evaluation RolloutState (re-fetched via Service.Get).
- **Status codes:** 200 OK; 404 NOT_FOUND (engine.ErrNotFound → "no rollout for this deployment"); 400 VALIDATION_FAILED (malformed verdict body, or a non-not-found evaluate error → "could not evaluate rollout").

8. recall_and_rollback_history_endpoints

Wire types: handlers_recall.go. Store seam: store.go (GetDeployment, UpdateDeployment, GetRelease, CreateDeployment, AppendRollback, ListRollbacks over the rollback_history table). The operator **recall** (server-driven rollback) surface from [../1.0.1-staged-rollout/rollback_ux.md §7](#) is now **WIRED** (it was planned in Revision 1). Recall is implemented as a **forward-fix**: it supersedes the current deployment and creates a NEW active deployment of the target release, recording a rollback_history row.

8.1 POST /api/v1/deployments/{deploymentId}/recall

Server-driven recall (rollback) of a deployment's current release to a previous-good release.

- **Roles:** operator, admin (server.go:157). **Maps to:** handlers_recall.go:handleRecall.
- **Request body** (RecallRequest):
 { "to_release_id": "...", "reason": "..." }. to_release_id is **required** — blank → 400 VALIDATION_FAILED (details: [{field:"to_release_id",issue:"required"}]). reason is optional.
- **Pre-checks (in order):** the deployment must exist (s.repo.GetDeployment) → else 404 NOT_FOUND ("deployment not found"); the deployment must have a current release (dep.ReleaseID non-empty) → else 400 VALIDATION_FAILED ("deployment has no current release to roll back from", details: [{field:"deployment",issue:"no current release"}]); the target release must exist (s.repo.GetRelease) → else 404 NOT_FOUND ("target release not found").
- **Effect (forward-fix, operator decision 2026-06-08):** the current deployment is updated to status="superseded" (UpdateDeployment); a new Deployment (status="active", target release, inheriting the prior deployment's strategy/group/target_count) is created (CreateDeployment); a RollbackRecord (kind="rollback") is appended (AppendRollback). The anti-downgrade invariant in handleClientUpdate guarantees a device is never offered a version ≤ its current, so the bootloader-enforced AVB anti-rollback is honored by construction.

- **Response 201** (RollbackView):

```
{
  "id": "ab12...",
  "deployment_id": "d12b...",
  "kind": "rollback",
  "from_release_id": "r-curr...",
  "to_release_id": "r-prev...",
  "recall_deployment_id": "d34c...",
  "reason": "high failure rate",
  "triggered_by": "operator@example.com",
  "details": { "mode": "forward-fix", "superseded_deployment":
    "d12b..." },
  "created_at": "2026-06-08T10:30:00Z"
}
```

- **Key fields** (RollbackView): id, deployment_id (the superseded/origin deployment), kind ("rollback"), from_release_id (the deployment's current release), to_release_id (the requested target), recall_deployment_id (the newly-created active deployment), reason, triggered_by (the caller subject from the token claims), details (string→string map; the build sets mode=forward-fix + superseded_deployment=<origin id>), created_at. The from/to/recall/reason/triggered_by/details fields are all omitempty.
- **Status codes:** 201 Created; 400 VALIDATION_FAILED (malformed body, blank to_release_id, or no current release); 404 NOT_FOUND (deployment or target release missing); 500 INTERNAL ("could not supersede current deployment", "could not create recall deployment", or "could not record rollback" on the respective store failures).
- **Audited:** a POST returning 2xx is logged by the audit middleware (§10) as a DEPLOYMENT_CREATE action (the route template ends in recall, which is not in the verb-refinement set; the resource is deployment).

8.2 GET /api/v1/deployments/{deploymentId}/rollbacks

Rollback/recall history for a deployment.

- **Roles:** viewer, operator, admin (server.go:158). **Maps to:** handlers_recall.go:handleListRollbacks.
- **Request:** no body, no query parameters consumed by the handler.
- **Response 200** (RollbackList): { "items": [RollbackView, ...] } — empty array, never null (the handler pre-allocates a non-nil slice). Items are returned in the store's ListRollbacks order. There is **no** 404 for an unknown deployment — an empty history is returned.
- **Status codes:** 200 OK; 500 INTERNAL ("could not list rollbacks" on store error).

9. delta_artifact_endpoints

Wire types: `handlers_delta.go`. Store seam: `store.go` (`GetArtifact`, `CreateDelta`, `FindDelta` over the `delta-artifact` table). Implements `delta_updates_design.md §3/§4` (register + look up a generated base→target delta payload).

9.1 POST /api/v1/deltas

Register a generated base→target delta artifact.

- **Roles:** operator, admin (`server.go:140`). **Maps to:** `handlers_delta.go:handleRegisterDelta`.
- **Request body** (`DeltaRegister`):

```
{
  "base_artifact_id": "a-base...",
  "target_artifact_id": "a-target...",
  "sha256": "9f86d0...",
  "size": 10485760,
  "storage_ref": "s3://helix-deltas/ab12.patch"
}
```

`base_artifact_id` and `target_artifact_id` are **required** (blank → 400 `VALIDATION_FAILED`, details: `[{field:"base_artifact_id",issue:"required"}, {field:"target_artifact_id",issue:"required"}]`) and must **differ** (equal → 400 `VALIDATION_FAILED`, details: `[{field:"target_artifact_id",issue:"must differ from base"}]`). `sha256`, `size`, and `storage_ref` are optional. - **Pre-checks:** both artifacts must exist (`s.repo.GetArtifact` for each) → else 404 `NOT_FOUND` ("artifact not found", details: `[{field:"<base|target>_artifact_id",issue:"unknown artifact"}]`). - **Response 201** (`DeltaView`): `{ "id", "base_artifact_id", "target_artifact_id", "sha256"(omitempty), "size"(omitempty), "storage_ref"(omitempty), "created_at" }`. - **Status codes:** 201 `Created`; 400 `VALIDATION_FAILED` (malformed body, missing ids, or `base==target`); 404 `NOT_FOUND` (unknown base/target artifact); 409 `CONFLICT` (`store.ErrConflict` → "a delta for this base→target pair already exists" — duplicate (base,target) pair); 500 `INTERNAL` ("could not register delta").

9.2 GET /api/v1/deltas

Look up the delta for a base/target pair.

- **Roles:** viewer, operator, admin (`server.go:141`). **Maps to:** `handlers_delta.go:handleFindDelta`.
- **Request:** no body. **Required** query parameters:
 - `?base=` — base artifact id. Blank → 400 `VALIDATION_FAILED` (details: `[{field:"base",issue:"required"}]`).
 - `?target=` — target artifact id. Blank → 400 `VALIDATION_FAILED` (details: `[{field:"target",issue:"required"}]`).
- **Response 200** (`DeltaView`, same shape as §9.1).

- **Status codes:** 200 OK; 400 VALIDATION_FAILED (missing base/target); 404 NOT_FOUND ("no delta for this base->target pair" — FindDelta returns an error).

10. audit_write_middleware_cross_cutting

The audit **write** path is not an endpoint but is the source of every GET /audit row, so it is recorded here for completeness (handlers_audit.go:auditMiddleware, wired on the protected group at server.go:130).

- **Placement (load-bearing):** mounted on the auth group **with** authMiddleware and **after** requireRole on each route, and writes **after** the handler (c.Next() then inspect c.Writer.Status()). So an RBAC-rejected (401/403) request is never audited, and only a 2xx mutating action is logged.
- **What is audited:** mutating methods only (POST/PUT/PATCH/DELETE, via isMutating) that returned 2xx. GETs and failed mutations are not written. GET /audit is itself not audited (it is a GET).
- **Action verb** (deriveAuditAction): <RESOURCE>_<VERB> SCREAMING_SNAKE_CASE derived from the gin route template (never the raw path, so ids never leak), e.g. GROUP_CREATE, GROUP_UPDATE, GROUP_DELETE, DEVICE_REGISTER, ARTIFACT_UPLOAD, and GROUP_MEMBER_* for the members sub-routes.
- **Best-effort:** AppendAudit errors are swallowed (_ =) — a failing audit sink never fails the user's already-successful request.

11. spec_vs_implementation_divergences

Per Constitution §11.4.6, where the prose specs and the built code disagree, the **code is authoritative** and the difference is stated here, never silently reconciled. Divergences that have since been closed in code are marked **RESOLVED** with the landing commit.

Audit (operational_endpoints.md §4 vs handlers_audit.go):

- The spec's actor is a nested object { user_id, subject }; the built AuditLogEntry.actor is a **flat string** (subject, falling back to user id).
- **RESOLVED (commit 028e656):** the audit handler now also consumes ?since and ?until (RFC3339; bad format → 400), implementing the spec's time-window filter (store.AuditFilter.Since/ .Until). The remaining gap is ?actor/?resource_id filtering, which is still **not** implemented; the built handler consumes ?action, ?resource_type, ?cursor, ?limit, ?since, ?until.
- The spec's response model is AuditList; the built type is AuditLogList.

Telemetry (operational_endpoints.md §5 vs handlers_telemetry.go):

- GET /devices/{id}/telemetry: the spec specifies pagination (?limit/?cursor), filters (?event_type, ?since, ?until, ?deployment_id), newest-first ordering, and an items array with id/success/duration_ms/bytes_transferred. The built handler takes **no query params**, returns the full history in **insertion order** under an **events** key, and the per-event view is {event, version, deployment_id, error_code, detail, timestamp,

- received_at} — no id, success, duration_ms, or bytes_transferred. It does **not** emit 404 for an unknown device (an empty history is returned).
- GET /telemetry/overview: the spec specifies a rich aggregate (scope, devices_total, devices_reporting, by_state latest-per-device, event_counts, failure_rate, generated_at) plus ?deployment_id/?os/?since/?until scoping. **RESOLVED (commit 028e656)**: the built response now adds failure_rate (float64, failure/(success+failure)) and by_state (string→int64 device-state map, from DeviceStateCounts) alongside event_counts
 - total (see §5.2). The remaining gap is the scope/devices_total/devices_reporting/generated_at fields and the ?deployment_id/?os/?since/?until query scoping, which are still **not** implemented.

Groups (operational_endpoints.md §6 vs handlers_group.go):

- The built GroupView is { id, name, description, member_count, created_at }. member_count is implemented (LANDED 4cb86d7). The id key is now group_id (RESOLVED — WIDENed to match the spec). filter_criteria is still **not** present (dynamic membership deferred — MVP is static-only).
- GET /groups has **no pagination** (no ?name/?limit/?cursor); the spec specifies a paginated GroupList. (Open — TRIM/WIDEN pending; low priority, groups are bounded.)
- POST /groups/{id}/members is a **batch** { "device_ids": [...] } returning 200 with a {added, already_member, not_found} result (RESOLVED — WIDENed to match the spec).
- GET /groups/{id}/members returns { group_id, device_ids: [string] }; the spec specifies an items array of { device_id, added_at } objects with pagination. (Open — needs a store membership-timestamp change; deferred to group WIDEN B2.)
- DELETE /groups/{id}/members/{deviceId} requires operator/admin in both spec and code (consistent). DELETE /groups/{id} requires admin in both (consistent).

Roles — consistent: the role assignments in the route table match operational_endpoints.md §3 for every operational route, and the rollout role assignments match rollout_engine.md §8 (create/evaluate operator/admin; get viewer+).

Rollout — consistent with the brick: RolloutState.status and RolloutDecision.action/ reason enumerate the exact ota-rollout-engine Status/Action/Reason string values; no values are invented. The handler maps deployment-missing → 404, plan-invalid → 400, and evaluate-not-found → 404.

Recall (rollback_ux.md §7 vs handlers_recall.go): LANDED (recall now wired; see §8). The endpoint is implemented as a **forward-fix** rather than an in-place N-1 re-flash: it supersedes the current deployment (status="superseded") and creates a NEW active deployment of the requested target release, recording a rollback_history row (kind="rollback", details.mode="forward-fix"). The caller supplies the explicit to_release_id (the build does not auto-resolve N-1). On success the response is 201 with the RollbackView record. GET /rollbacks returns the full history and does **not** 404 on an unknown deployment (empty list).